Configurations Page — Info Doc

Page Overview

Purpose

The Configurations page is the management surface for named, reusable parameter presets that feed the Deep Dive, Deep Sweep, and Routines pages. A configuration is a saved bundle of search parameters (repository or repository set, keywords, per-repository result cap, AI toggle, AI settings, email toggles, storage settings, and — for routines — schedule and execution-location fields). Date ranges are intentionally not persisted in configurations; they are set fresh per run or per routine fire.

Primary User Flows

1. Open the page and land on the **Routine Configs** tab by default; switch to **Manual Configs** to view manual dive and manual sweep presets together.
2. Select one or more rows via per-row checkboxes or the select-all checkbox in the header.
3. Click **Export Selected** to write the chosen rows to a ZIP archive of JSON files on disk; a success banner shows the path and (on desktop) a Reveal button that opens the containing folder.
4. Click **Import** to pick one or more `.json` files through the native file picker; each file is validated and inserted as a new configuration.
5. Click **Delete Selected** to open a confirmation dialog; confirming deletes every selected configuration, and for rows whose `config_type` is `routine` the linked scheduled routine is also deleted in the same operation.

Inputs and Outputs

- **Inputs**: selection state in the table, JSON files chosen through the import file picker, and the IDs of selected rows sent in the export request body.
- **Outputs**: the exported ZIP archive on disk (path returned to the UI), success and error banners, and refreshed table state after mutations.

Known Constraints

- Configuration names are not uniquely constrained by the database schema; rows are keyed by auto-increment integer `id`.

- The `config_type` column is restricted by a CHECK constraint to `manual_dive`, `manual_sweep`, or `routine`.
- Imports only accept files whose filename ends in `.json`; any non-JSON file short-circuits the entire batch with an error.
- Deleting a routine configuration cascades to the linked routine row when the stored parameters contain a valid `linked_routine_id` pointing at an existing routine.
- The Reveal button on the export success banner only appears when the Electron preload has exposed `window.resmonAPI.revealPath`; the button label switches to "Reveal in Finder" on macOS (`platform === 'darwin'`) and "Reveal in File Explorer" otherwise.

Frontend

Route and Main Component

- Route: `/configurations` (registered in `App.tsx`).
- Main component: `resmon_scripts/frontend/src/pages/ConfigurationsPage.tsx`.

Child Components and Hooks

- `PageHelp` — collapsible help panel with `storageKey="configurations"` and three sections (configuration definition, tabs explanation, import/export round-trip).
- `apiClient` — shared wrapper used for `GET /api/configurations`, `POST /api/configurations/export`, and `DELETE /api/configurations/{id}`.
- A raw `fetch` call against `getBaseUrl()` is used for `POST /api/configurations/import` because the payload is a `multipart/form-data` `FormData` body rather than JSON.
- `window.resmonAPI` (Electron preload) supplies `getBackendPort()`, `platform`, and `revealPath(path)`.
- React hooks: `useState`, `useEffect`, `useCallback`, `useRef` for the hidden file input.

UI State Model

Local state variables held in `ConfigurationsPage`:

- `configs: Config[]` — the full list fetched from `/api/configurations`; `parameters` may arrive as an object or a string depending on the row.
- `loading: boolean` — true until the initial fetch resolves; gates the loading placeholder.
- `tab: 'routine' | 'manual'` — active tab. `routine` filters to `config_type === 'routine'`; `manual` filters to `manual_dive` or `manual_sweep`.

- `selected: Set<number>` — set of selected configuration IDs; toggled per-row via `handleToggle` and in bulk via `handleToggleAll`.
- `error: string` — inline error banner content.
- `status: string` — inline success banner content (e.g., export path, import count); auto-cleared on timers.
- `exportPath: string` — captured path from the most recent export; used to drive the Reveal button and cleared on the 10-second timer alongside `status`.
- `confirmDelete: boolean` — toggles the delete confirmation overlay; the overlay text counts how many of the selected rows are `routine` configs so the user sees the cascade impact.
- `fileRef: RefObject<HTMLInputElement>` — hidden file input used to open the native file picker for import.

Derived values:

- `filtered` — `configs` filtered by the current tab.
- `allSelected` — true when every visible row is selected (drives the header checkbox state).

Key Interactions and Events

- **Fetch on mount** (`fetchConfigs`): `GET /api/configurations` → `setConfigs` ; also called after successful import and after delete to refresh.
- **Tab switch**: flips `tab` between `routine` and `manual`.
- **Row checkbox** (`handleToggle`): mutates the `selected` set.
- **Header checkbox** (`handleToggleAll`): clears the set if `allSelected` , otherwise inserts every ID in `filtered`.
- **Badge rendering** (`configTypeBadgeClass`): maps `manual_dive` / `deep_dive` / `dive` → `badge-type-dive` (pink-purple), `manual_sweep` / `deep_sweep` / `sweep` → `badge-type-sweep` (green-teal), `routine` → `badge-type-config-routine` (amber/gold), any other value → `badge-type-other`. The `badge-type-config-routine` class is intentionally distinct from `badge-type-routine` , which is used elsewhere in the app (Dashboard, Results & Logs) to mark routine-fired executions; this keeps saved routine configs visually separate from routine executions in the shared row components.
- **Export** (`handleExport`): no-op when the selection is empty. Posts `{ ids: Array.from(selected) }` to `/api/configurations/export` ; on success stores `resp.path` in both `exportPath` and `status` , then clears both after 10 s.
- **Reveal** (`handleReveal`): calls `window.resmonAPI.revealPath(exportPath)` when both `exportPath` and the preload API are available. Button label is computed once per render via `revealLabel`.

- **Import** (`handleImport`): iterates `e.target.files` , rejects any file whose name does not end in `.json` with an inline error and a cleared file input. Builds a `FormData` with key `files` per file and POSTs to `/api/configurations/import` . On success reads `data.imported` (and the new `data.routines_created` counter described below) and refreshes the list. The file input value is always reset at the end so the same file can be re-selected.
- **Edit** (`handleEditSave`): the per-row `Edit` button opens a config-type-aware modal with three variants. For `routine` rows the modal pre-populates the full Routines-style form (name + cron + multi-repo selector + date range + keywords + max-results + AI / Email / Email-AI / Notify toggles + Execution-Location radio + the shared `AIOverridePanel`) and saves via `PUT /api/routines/{linked_routine_id}` ; on a `404` the handler falls back to `PUT /api/configurations/{id}` with the full routine payload (with `linked_routine_id` cleared) so an orphaned routine config — e.g. one imported from another machine before OOB-11 landed, or one whose linked routine was deleted out-of-band — can still be edited in place. For `manual_sweep` rows the modal renders the same form minus cron and minus the routine-only toggles and saves via `PUT /api/configurations/{id}` . For `manual_dive` rows the modal renders the same form as `manual_sweep` but with `RepositorySelector mode="single"` . After every successful save the handler calls `notifyConfigurationsChanged()` so sibling `ConfigLoader` s and the Routines page refetch. For the `routine` variant specifically, the same cross-page invalidation is also reachable via the Routines page's modal and the Calendar page popover's `Edit Routine` button — both of which use the shared `RoutineEditModal` component, which broadcasts on both the configurations bus and the routines bus (`notifyRoutinesChanged()`) on save. Either path keeps this page's row list and the Routines page's row list in lockstep without a manual reload.
- **View JSON** (`handleViewJson`): the per-row `View JSON` button opens a read-only modal that renders the row's full payload (including its `parameters` JSON, computed `display_name` , and any `linked_routine_id`) inside a `<pre>` block. The modal exposes a Copy-to-clipboard button (uses `navigator.clipboard.writeText` with a `document.execCommand('copy')` fallback) and a Close button; no fetch, mutation, or bus broadcast happens on this path. The button is intended for inspection, debugging, and supporting an exported config side-by-side with its in-app representation.
- **Delete confirm flow** (`handleDeleteSelected`): iterates selected IDs and issues `DELETE /api/configurations/{id}` for each, swallowing per-row errors so the loop continues. Afterwards it clears `selected` , closes the confirm overlay, and re-fetches.

Error and Empty States

- Initial load: renders "Loading configurations…" inside `page-content` while `loading` is true.
- Empty tab: when `filtered` is empty the table body shows a muted placeholder row.

- Fetch failure: the thrown error's message is placed in the `error` banner above the tab bar.
- Import failure: failed `fetch` or non-OK responses surface a prefixed "Import failed: …" error; both `status` and `error` are auto-cleared after 5 s at the end of the handler.
- Invalid file type during import: a specific "Invalid file type: . Only .json files accepted." error is shown and the batch is aborted before any upload.

Backend

API Endpoints

Defined in `resmon_scripts/resmon.py` under the "Configurations" section:

- `GET /api/configurations` — list all configurations (optional `config_type` query filter); `parameters` is JSON-decoded into a dict per row when possible.
- `POST /api/configurations` (201) — create a configuration from a `ConfigCreate` body (`name` , `config_type` , `parameters`); returns `{id, name, config_type}` .
- `PUT /api/configurations/{config_id}` — update a configuration from a `ConfigUpdate` body (`name?` , `parameters?`); 404s if the ID is unknown; `parameters` is re-serialized to JSON before the write.
- `DELETE /api/configurations/{config_id}` — delete a configuration; 404s if the ID is unknown; cascades to `delete_routine(conn, rid)` when the row's `config_type` is `routine` and its parameters contain an integer `linked_routine_id` resolving to an existing routine.
- `POST /api/configurations/export` — body is `ConfigExport` (`ids: list[int]`); writes a ZIP archive via `config_manager.export_configs` and returns `{"path": str}` .
- `POST /api/configurations/import` — accepts a multipart `files: list[UploadFile]` ; each upload is written to a temp `.json` file and passed to `config_manager.import_configs` . The handler then post-processes the imported rows: for every config whose `config_type == 'routine'` it parses the inner `parameters` JSON, builds a routine insert payload, **forces `is_active=0`** so imported routines are deactivated by default and bulk imports never auto-fire, calls `insert_routine` , captures the new routine id, and rewrites the just-imported config row's `parameters` JSON so `linked_routine_id` points at the new routine. This restores the bidirectional Routines ↔ routine-configs invariant — every routine config has a real routine row and vice-versa — that the rest of the app already enforces in both other directions (`_sync_routine_config` on routine create/update, and the `delete_routine` cascade on `DELETE /api/configurations/{id}`). Returns `{"imported": n, "routines_created": m, "errors": []}` ; per-file JSON-decode failures and `validate_config` failures are still logged server-side and skipped silently, so `imported` may be

less than the number of uploaded files, and per-row routine-creation failures (logged and skipped) may make `routines_created` less than the number of imported routine configs.

Request/Response Patterns

- List response: array of `{id, name, config_type, parameters, created_at, updated_at}` objects. `parameters` is returned as a parsed object when the stored JSON is valid, otherwise as the raw string.
- Create response: minimal `{id, name, config_type}` echo.
- Update response: `{id, ...updates}` echoing only the fields actually touched.
- Delete response: `{success: True}` .
- Export response: `{path}` pointing either at `<export_directory>/resmon_configs_<YYYYMMDDTHHMMSS>.zip` when the `export_directory` setting is populated, or at a `tempfile.NamedTemporaryFile(suffix=".zip")` path when it is not.
- Import response: `{imported: int, errors: []}` . The current implementation always returns an empty `errors` array; per-file JSON-decode failures and `validate_config` failures are logged server-side and those files are skipped silently, so `imported` may be less than the number of uploaded files.

Persistence Touchpoints

- SQLite table `saved_configurations` (`id` , `name` , `config_type` , `parameters` , `created_at` , `updated_at`) — the only table directly written by these endpoints. `config_type` is constrained to `manual_dive` , `manual_sweep` , or `routine` at the schema level.
- `routines` table — read on delete to check whether a linked routine exists, and written via `delete_routine` when a routine config is deleted.
- `app_settings` — `export_directory` is read to decide where to write the exported ZIP.

Execution Side Effects

- Export writes a ZIP archive to disk. Each member is a JSON file named `config_<config_type>_<slug>_<id>.json` with the shape `{config_type, name, ...parameters}` .
- Import validates each parsed file against `CONFIG_SCHEMA` (required keys `config_type` , `name` , `repositories` , `keywords` ; enum-checked `config_type` ; type-checked arrays/objects/integers with their minimums and maximums) before calling `save_config` , which re-validates and inserts.

- Deleting a `routine` config cascades to the scheduled routine row via `delete_routine(conn, rid)` ; this indirectly triggers the scheduler-sync helpers that mirror routine changes to APScheduler, so the scheduled job is removed as well.
- No SSE, email, or cloud-sync side effects originate on this page; configurations are a local-only concept in the current implementation.

Configurations Page — Info Doc

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Backend

API Endpoints

Request/Response Patterns

Persistence Touchpoints

Execution Side Effects